

27.mobilenet模型部署

1. 编译程序

程序执行命令

代码块

```
1  #!/bin/bash
2  set -e
3  export PATH=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-
  gnu/bin:$PATH
4  rm -rf build
5
6  cmake -S . -B build \
7    -DCMAKE_SYSTEM_NAME=Linux \
8    -DCMAKE_CXX_COMPILER=aarch64-rockchip1031-linux-gnu-g++
9
10 cmake --build build -j4
11 cmake --install build
12 echo "✅ 打包完成! 部署目录: $PWD/install"
```

CMakeLists.txt

代码块

```
1  # 设置最低版本号
2  cmake_minimum_required(VERSION 3.11 FATAL_ERROR)
3  # 设置项目名称
4  project(rk3588-mobilenet VERSION 0.0.1 LANGUAGES CXX)
5
6  # 输出系统信息
7  message(STATUS "System: ${CMAKE_SYSTEM_NAME} ${CMAKE_SYSTEM_VERSION}")
8
9  # 设置编译器
10 set(CMAKE_CXX_STANDARD 14)
11 set(CMAKE_CXX_STANDARD_REQUIRED ON)
12
13 # 设置库架构
14 set(LIB_ARCH "aarch64")
15 set(DEVICE_NAME "RK3588")
16
17 # rknn_api 文件夹路径
```

```

18 set(RKNN_API_PATH ${CMAKE_CURRENT_SOURCE_DIR}/librknn_api)
19 # rknn_api include 路径
20 set(RKNN_API_INCLUDE_PATH ${RKNN_API_PATH}/include)
21 # rknn_api lib 路径
22 set(RKNN_API_LIB_PATH ${RKNN_API_PATH}/${LIB_ARCH}/librknnrt.so)
23
24 # 寻找OpenCV库，使用自定义的OpenCV_DIR
25 set(3RDPARTY_PATH ${CMAKE_CURRENT_SOURCE_DIR}/3rdparty)
26 set(OpenCV_DIR ${3RDPARTY_PATH}/opencv/opencv-linux-${LIB_ARCH}/share/OpenCV)
27 find_package(OpenCV REQUIRED)
28 # 输出OpenCV信息
29 message(STATUS "    include path: ${OpenCV_INCLUDE_DIRS}")
30
31 # 用来搜索头文件的目录
32 include_directories(
33     ${OpenCV_INCLUDE_DIRS}
34     ${RKNN_API_INCLUDE_PATH}
35 )
36
37 # 测试NPU: rknn mobilenet
38 add_executable(mobilenet src/mobilenet_demo.cpp)
39
40 # 链接库
41 target_link_libraries(mobilenet
42     ${RKNN_API_LIB_PATH}
43     ${OpenCV_LIBS}
44 )
45
46 # ===== 安装配置 =====
47 set(CMAKE_INSTALL_PREFIX ${CMAKE_SOURCE_DIR}/install/${PROJECT_NAME})
48 set(CMAKE_SKIP_BUILD_RPATH FALSE)
49 set(CMAKE_BUILD_WITH_INSTALL_RPATH TRUE)
50 set(CMAKE_INSTALL_RPATH "$ORIGIN/../lib")
51
52 # 安装规则
53 install(TARGETS mobilenet DESTINATION .)
54 install(FILES ${RKNN_API_LIB_PATH} DESTINATION lib)
55 install(DIRECTORY weights/ DESTINATION model)
56 install(DIRECTORY images/ DESTINATION model)

```

查看结果

```

root@topeet:/userdata/rk3588-mobilenet-v1# ls
lib mobilenet model
root@topeet:/userdata/rk3588-mobilenet-v1# ./mobilenet ./model/mobilenet_v1.rknn ./model/dog_224x224.jpg
model input num: 1, output num: 1
input tensors:
  index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], n_elems=150528, size=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE
  zp=0, scale=0.007812
output tensors:
  index=0, name=MobilenetV1/Predictions/Reshape_1, n_dims=2, dims=[1, 1001, 0, 0], n_elems=1001, size=2002, fmt=UNDEFINED
  type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000
rknn_run
--- Top5 ---
156: 0.884766
155: 0.054016
205: 0.003677
284: 0.002974
285: 0.000189
root@topeet:/userdata/rk3588-mobilenet-v1#

```

2. 源码分析

代码块

```

1  #include "opencv2/core/core.hpp"
2  #include "opencv2/imgcodecs.hpp"
3  #include "opencv2/imgproc.hpp"
4  #include "rknn_api.h"
5
6  #include <stdint.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <sys/time.h>
10
11 #include <fstream>
12 #include <iostream>
13
14 using namespace std;
15 using namespace cv;
16
17 /*-----
18                      Functions
19 -----*/
20
21 static void dump_tensor_attr(rknn_tensor_attr* attr)
22 {
23     printf("  index=%d, name=%s, n_dims=%d, dims=[%d, %d, %d, %d], n_elems=%d,
24     size=%d, fmt=%s, type=%s, qnt_type=%s, "
25     "zp=%d, scale=%f\n",
26     attr->index, attr->name, attr->n_dims, attr->dims[0], attr->dims[1],
27     attr->dims[2], attr->dims[3],
28     attr->n_elems, attr->size, get_format_string(attr->fmt),
29     get_type_string(attr->type),
30     get_qnt_type_string(attr->qnt_type), attr->zp, attr->scale);
31 }
32
33 static unsigned char* load_model(const char* filename, int* model_size)

```

```

31  {
32      FILE* fp = fopen(filename, "rb");
33      if (fp == nullptr) {
34          printf("fopen %s fail!\n", filename);
35          return NULL;
36      }
37      fseek(fp, 0, SEEK_END);
38      int model_len = ftell(fp);
39      unsigned char* model = (unsigned char*)malloc(model_len);
40      fseek(fp, 0, SEEK_SET);
41      if (model_len != fread(model, 1, model_len, fp)) {
42          printf("fread %s fail!\n", filename);
43          free(model);
44          return NULL;
45      }
46      *model_size = model_len;
47      if (fp) {
48          fclose(fp);
49      }
50      return model;
51  }
52
53  static int rknn_GetTop(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass,
54  uint32_t outputCount, uint32_t topNum)
55  {
56
57      #define MAX_TOP_NUM 20
58      if (topNum > MAX_TOP_NUM)
59          return 0;
60
61      memset(pfMaxProb, 0, sizeof(float) * topNum);
62      memset(pMaxClass, 0xff, sizeof(float) * topNum);
63
64      for (j = 0; j < topNum; j++) {
65          for (i = 0; i < outputCount; i++) {
66              if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
67  (pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
68  (i == *(pMaxClass + 4))) {
69                  continue;
70              }
71
72              if (pfProb[i] > *(pfMaxProb + j)) {
73                  *(pfMaxProb + j) = pfProb[i];
74                  *(pMaxClass + j) = i;
75              }

```

```

76     }
77
78     return 1;
79 }
80
81 /*-----
82                                     Main Function
83 -----*/
84 int main(int argc, char** argv)
85 {
86     const int MODEL_IN_WIDTH      = 224;
87     const int MODEL_IN_HEIGHT     = 224;
88     const int MODEL_IN_CHANNELS   = 3;
89
90     rknn_context ctx = 0;
91     int          ret;
92     int          model_len = 0;
93     unsigned char* model;
94
95     const char* model_path = argv[1];
96     const char* img_path   = argv[2];
97
98     if (argc != 3) {
99         printf("Usage: %s <rknn model> <image_path> \n", argv[0]);
100        return -1;
101    }
102
103    // Load image
104    cv::Mat orig_img = imread(img_path, cv::IMREAD_COLOR);
105    if (!orig_img.data) {
106        printf("cv::imread %s fail!\n", img_path);
107        return -1;
108    }
109
110    cv::Mat orig_img_rgb;
111    //rknn模型说明来源于RKNN-Toolkit2的examples/tflite/mobilenet_v1示例，输入通道顺序与python代码保持一致
112    cv::cvtColor(orig_img, orig_img_rgb, cv::COLOR_BGR2RGB);
113
114    cv::Mat img = orig_img_rgb.clone();
115    if (orig_img.cols != MODEL_IN_WIDTH || orig_img.rows != MODEL_IN_HEIGHT) {
116        printf("resize %d %d to %d %d\n", orig_img.cols, orig_img.rows,
MODEL_IN_WIDTH, MODEL_IN_HEIGHT);
117        cv::resize(orig_img_rgb, img, cv::Size(MODEL_IN_WIDTH, MODEL_IN_HEIGHT),
0, 0, cv::INTER_LINEAR);
118    }
119

```

```

120 // Load RKNN Model
121 model = load_model(model_path, &model_len);
122 ret = rknn_init(&ctx, model, model_len, 0, NULL);
123 if (ret < 0) {
124     printf("rknn_init fail! ret=%d\n", ret);
125     return -1;
126 }
127
128 // Get Model Input Output Info
129 rknn_input_output_num io_num;
130 ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
131 if (ret != RKNN_SUCC) {
132     printf("rknn_query fail! ret=%d\n", ret);
133     return -1;
134 }
135 printf("model input num: %d, output num: %d\n", io_num.n_input,
io_num.n_output);
136
137 printf("input tensors:\n");
138 rknn_tensor_attr input_attrs[io_num.n_input];
139 memset(input_attrs, 0, sizeof(input_attrs));
140 for (int i = 0; i < io_num.n_input; i++) {
141     input_attrs[i].index = i;
142     ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &
(input_attrs[i]), sizeof(rknn_tensor_attr));
143     if (ret != RKNN_SUCC) {
144         printf("rknn_query fail! ret=%d\n", ret);
145         return -1;
146     }
147     dump_tensor_attr(&(input_attrs[i]));
148 }
149
150 printf("output tensors:\n");
151 rknn_tensor_attr output_attrs[io_num.n_output];
152 memset(output_attrs, 0, sizeof(output_attrs));
153 for (int i = 0; i < io_num.n_output; i++) {
154     output_attrs[i].index = i;
155     ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &
(output_attrs[i]), sizeof(rknn_tensor_attr));
156     if (ret != RKNN_SUCC) {
157         printf("rknn_query fail! ret=%d\n", ret);
158         return -1;
159     }
160     dump_tensor_attr(&(output_attrs[i]));
161 }
162
163 // Set Input Data

```

```
164     rknn_input inputs[1];
165     memset(inputs, 0, sizeof(inputs));
166     inputs[0].index = 0;
167     inputs[0].type = RKNN_TENSOR_UINT8;
168     inputs[0].size = img.cols * img.rows * img.channels() * sizeof(uint8_t);
169     inputs[0].fmt = RKNN_TENSOR_NHWC;
170     inputs[0].buf = img.data;
171
172     ret = rknn_inputs_set(ctx, io_num.n_input, inputs);
173     if (ret < 0) {
174         printf("rknn_input_set fail! ret=%d\n", ret);
175         return -1;
176     }
177
178     // Run
179     printf("rknn_run\n");
180     ret = rknn_run(ctx, nullptr);
181     if (ret < 0) {
182         printf("rknn_run fail! ret=%d\n", ret);
183         return -1;
184     }
185
186     // Get Output
187     rknn_output outputs[1];
188     memset(outputs, 0, sizeof(outputs));
189     outputs[0].want_float = 1;
190     ret = rknn_outputs_get(ctx, 1, outputs, NULL);
191     if (ret < 0) {
192         printf("rknn_outputs_get fail! ret=%d\n", ret);
193         return -1;
194     }
195
196     // Post Process
197     for (int i = 0; i < io_num.n_output; i++) {
198         uint32_t MaxClass[5];
199         float fMaxProb[5];
200         float* buffer = (float*)outputs[i].buf;
201         uint32_t sz = outputs[i].size / 4;
202
203         rknn_GetTop(buffer, fMaxProb, MaxClass, sz, 5);
204
205         printf(" --- Top5 ---\n");
206         for (int i = 0; i < 5; i++) {
207             printf("%3d: %8.6f\n", MaxClass[i], fMaxProb[i]);
208         }
209     }
210
```

```

211 // Release rknn_outputs
212 rknn_outputs_release(ctx, 1, outputs);
213
214 // Release
215 if (ctx > 0)
216 {
217     rknn_destroy(ctx);
218 }
219 if (model) {
220     free(model);
221 }
222 return 0;
223 }
224

```

总体流程



1. 加载模型 → rknn_init()
2. 查询输入输出信息 → rknn_query()
3. 预处理 → imread()+resize + BGR2RGB
4. 设置输入 → rknn_inputs_set()
5. 推理 → rknn_run()
6. 获取输出 → rknn_outputs_get()
7. 后处理 → rknn_GetTop()
8. 释放资源

2.1 导入依赖库

代码块

```

1  #include "opencv2/core/core.hpp" // OpenCV 核心功能: Mat 类、数据类型等
2  #include "opencv2/imgcodecs.hpp" // 图像读取/保存
3  #include "opencv2/imgproc.hpp"  // 图像处理 (如 cvtColor, resize)
4  #include "rknn_api.h"           // RKNN SDK 接口, 用于加载模型、推理等
5
6  #include <stdint.h>              // 定义标准整数类型 (如 uint32_t)
7  #include <stdio.h>              // C 标准库: 输入输出、内存管理
8  #include <stdlib.h>             // C 标准库: 输入输出、内存管理
9  #include <sys/time.h>           // 时间相关
10
11 #include <fstream>              // C++ 输入流

```


2.2 辅助函数

2.2.1 dump_tensor_attr

代码块

```
1  static void dump_tensor_attr(rknn_tensor_attr* attr)
2  {
3      printf("  index=%d, name=%s, n_dims=%d, dims=[%d, %d, %d, %d], n_elems=%d,
4      size=%d, fmt=%s, type=%s, qnt_type=%s, "
5      "zp=%d, scale=%f\n",
6      attr->index, attr->name, attr->n_dims, attr->dims[0], attr->dims[1],
7      attr->dims[2], attr->dims[3],
8      attr->n_elems, attr->size, get_format_string(attr->fmt),
9      get_type_string(attr->type),
10     get_qnt_type_string(attr->qnt_type), attr->zp, attr->scale);
11 }
```

打印 RKNN 模型的输入/输出张量属性信息，便于调试。

- attr->index: 张量索引（第几个输入/输出）
- attr->name: 张量名称（如 "input" 或 "output_0"）
- attr->dims[]: 维度（NCHW 或 NHWC）
- attr->fmt: 数据格式（如 RKNN_TENSOR_NHWC）
- attr->type: 数据类型（RKNN_TENSOR_UINT8, RKNN_TENSOR_FLOAT32）
- attr->qnt_type: 量化类型（RKNN_QUANTIZE_NONE, RKNN_QUANTIZE_INT8 等）
- attr->zp, attr->scale: 量化参数（仅用于 INT8 模型）

2.2.2 load_model

代码块

```
1  static unsigned char* load_model(const char* filename, int* model_size)
2  {
3      FILE* fp = fopen(filename, "rb");
4      if (fp == nullptr) {
5          printf("fopen %s fail!\n", filename);
6          return NULL;
7      }
8      fseek(fp, 0, SEEK_END);
```

```

9      int          model_len = ftell(fp);
10     unsigned char* model     = (unsigned char*)malloc(model_len);
11     fseek(fp, 0, SEEK_SET);
12     if (model_len != fread(model, 1, model_len, fp)) {
13         printf("fread %s fail!\n", filename);
14         free(model);
15         return NULL;
16     }
17     *model_size = model_len;
18     if (fp) {
19         fclose(fp);
20     }
21     return model;
22 }

```

从磁盘加载 .rknn 模型文件到内存中。步骤如下：

1. 打开文件为二进制模式（"rb"）
2. 跳到文件末尾，获取文件大小（ftell()）
3. 分配内存缓冲区（malloc()）
4. 回到文件开头，读取全部内容（fread()）
5. 返回指针和大小

2.2.3 rknn_GetTop

代码块

```

1  static int rknn_GetTop(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass,
2  uint32_t outputCount, uint32_t topNum)
3  {
4      uint32_t i, j;
5      #define MAX_TOP_NUM 20
6      if (topNum > MAX_TOP_NUM)
7          return 0;
8
9      memset(pfMaxProb, 0, sizeof(float) * topNum);
10     memset(pMaxClass, 0xff, sizeof(float) * topNum);
11
12     for (j = 0; j < topNum; j++) {
13         for (i = 0; i < outputCount; i++) {
14             if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
15 (pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
16 (i == *(pMaxClass + 4))) {

```

```

16         continue;
17     }
18
19     if (pfProb[i] > *(pfMaxProb + j)) {
20         *(pfMaxProb + j) = pfProb[i];
21         *(pMaxClass + j) = i;
22     }
23 }
24 }
25
26 return 1;
27 }

```

从分类概率数组中提取 Top-N 的类别及其置信度。

- pfProb: 模型输出的概率向量（如 [0.9, 0.1, 0.05, ...]）
- pfMaxProb: 输出：Top-N 的最大概率值
- pMaxClass: 输出：Top-N 对应的类别索引
- outputCount: 总类别数（如 1000）
- topNum: 要提取的个数（如 5）

2.3 主程序

2.3.1 常量定义

代码块

```

1  const int MODEL_IN_WIDTH    = 224;
2  const int MODEL_IN_HEIGHT   = 224;
3  const int MODEL_IN_CHANNELS = 3;

```

模型输入尺寸（宽、高、通道数），必须与训练时一致。

MobileNet 输入：224×224×3（RGB 图像）

2.3.2 初始化 RKNN 对象

代码块

```

1  rknn_context ctx = 0;
2  int ret;

```

- ctx: RKNN 上下文句柄，用于后续操作
- ret: 存储 API 返回值，判断是否成功

2.3.3 加载模型文件

代码块

```
1  const char* model_path = argv[1];
2  const char* img_path   = argv[2];
3
4  if (argc != 3) {
5      printf("Usage: %s <rknn model> <image_path> \n", argv[0]);
6      return -1;
7  }
8
9  model = load_model(model_path, &model_len);
10 ret    = rknn_init(&ctx, model, model_len, 0, NULL);
11 if (ret < 0) {
12     printf("rknn_init fail! ret=%d\n", ret);
13     return -1;
14 }
```

功能:

- 获取命令行参数：模型路径 + 图像路径
- 调用 `load_model()` 加载 `.rknn` 文件
- 调用 `rknn_init()` 初始化 RKNN 上下文

2.3.4 获取模型输入输出信息

代码块

```
1  rknn_input_output_num io_num;
2  ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
3  if (ret != RKNN_SUCC) {
4      printf("rknn_query fail! ret=%d\n", ret);
5      return -1;
6  }
7  printf("model input num: %d, output num: %d\n", io_num.n_input,
        io_num.n_output);
```

查询模型的输入输出数量，用于后续设置输入/获取输出

示例: model input num: 1, output num: 1

2.3.5 查询输入张量属性

代码块

```
1  rknn_tensor_attr input_attrs[io_num.n_input];
2  memset(input_attrs, 0, sizeof(input_attrs));
3  for (int i = 0; i < io_num.n_input; i++) {
4      input_attrs[i].index = i;
5      ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &
        (input_attrs[i]), sizeof(rknn_tensor_attr));
6      if (ret != RKNN_SUCC) {
7          printf("rknn_query fail! ret=%d\n", ret);
8          return -1;
9      }
10     dump_tensor_attr(&(input_attrs[i]));
11 }
```

获取每个输入张量的详细信息（维度、格式、类型等），并打印出来。

示例： index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], fmt=NHWC, type=UINT8, ...

2.3.6 查询输出张量属性

代码块

```
1  rknn_tensor_attr output_attrs[io_num.n_output];
2  memset(output_attrs, 0, sizeof(output_attrs));
3  for (int i = 0; i < io_num.n_output; i++) {
4      output_attrs[i].index = i;
5      ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &
        (output_attrs[i]), sizeof(rknn_tensor_attr));
6      if (ret != RKNN_SUCC) {
7          printf("rknn_query fail! ret=%d\n", ret);
8          return -1;
9      }
10     dump_tensor_attr(&(output_attrs[i]));
11 }
```

同上，但针对输出张量。

对于分类模型，输出通常是 `[1, N]`，其中 N 是类别数。

2.3.7 图像预处理

代码块

```
1  cv::Mat orig_img = imread(img_path, cv::IMREAD_COLOR);
```

```

2  if (!orig_img.data) {
3      printf("cv::imread %s fail!\n", img_path);
4      return -1;
5  }
6
7  cv::Mat orig_img_rgb;
8  cv::cvtColor(orig_img, orig_img_rgb, cv::COLOR_BGR2RGB);
9
10 cv::Mat img = orig_img_rgb.clone();
11 if (orig_img.cols != MODEL_IN_WIDTH || orig_img.rows != MODEL_IN_HEIGHT) {
12     printf("resize %d %d to %d %d\n", orig_img.cols, orig_img.rows,
13         MODEL_IN_WIDTH, MODEL_IN_HEIGHT);
14     cv::resize(orig_img_rgb, img, cv::Size(MODEL_IN_WIDTH, MODEL_IN_HEIGHT), 0,
15         0, cv::INTER_LINEAR);
16 }

```

将图像转换为模型所需的输入格式。

步骤详解：

1. 读取图像（BGR 格式）
2. 转换为 RGB（RKNN 模型通常期望 RGB）
3. 如果尺寸不匹配，缩放到 224×224
4. 使用双线性插值（`INTER_LINEAR`）保持质量

✅ 注意：`cv::imread` 默认是 BGR，而神经网络训练时用的是 RGB，所以必须转换

2.3.8 设置输入数据

代码块

```

1  rknn_input inputs[1];
2  memset(inputs, 0, sizeof(inputs));
3  inputs[0].index = 0;
4  inputs[0].type = RKNN_TENSOR_UINT8;
5  inputs[0].size = img.cols * img.rows * img.channels() * sizeof(uint8_t);
6  inputs[0].fmt = RKNN_TENSOR_NHWC;
7  inputs[0].buf = img.data;
8
9  ret = rknn_inputs_set(ctx, io_num.h_input, inputs);
10 if (ret < 0) {
11     printf("rknn_input_set fail! ret=%d\n", ret);
12     return -1;
13 }

```

将图像数据包装成 RKNN 可识别的输入结构，并设置到上下文中。

字段说明：

- `index` : 输入索引（从 0 开始）
- `type` : 数据类型（`UINT8` 表示 0~255 的像素值）
- `size` : 数据总字节数
- `fmt` : 数据格式（`NHWC` : N=Batch, H=Height, W=Width, C=Channel）
- `buf` : 数据指针（指向 OpenCV Mat 的原始数据）

2.3.9 模型推理

代码块

```
1  printf("rknn_run\n");
2  ret = rknn_run(ctx, nullptr);
3  if (ret < 0) {
4      printf("rknn_run fail! ret=%d\n", ret);
5      return -1;
6  }
```

执行模型推理（前向传播）。

- `rknn_run()` 是真正的“推理”调用
- 第二个参数为 `nullptr`，表示使用默认配置
- 成功后，结果会写入输出缓冲区

2.3.10 获取输出结果

代码块

```
1  rknn_output outputs[1];
2  memset(outputs, 0, sizeof(outputs));
3  outputs[0].want_float = 1;
4  ret = rknn_outputs_get(ctx, 1, outputs, NULL);
5  if (ret < 0) {
6      printf("rknn_outputs_get fail! ret=%d\n", ret);
7      return -1;
8  }
```

从 RKNN 内核中获取推理结果。

- `outputs[0].want_float = 1` : 要求输出为浮点数（即使模型是 INT8 量化）

- rknn_outputs_get() 将结果复制到用户空间
- outputs[0].buf 指向输出数据 (float 数组)

2.3.11 后处理

代码块

```
1  for (int i = 0; i < io_num.n_output; i++) {
2      uint32_t MaxClass[5];
3      float    fMaxProb[5];
4      float*   buffer = (float*)outputs[i].buf;
5      uint32_t sz      = outputs[i].size / 4;
6
7      rknn_GetTop(buffer, fMaxProb, MaxClass, sz, 5);
8
9      printf(" --- Top5 ---\n");
10     for (int i = 0; i < 5; i++) {
11         printf("%3d: %8.6f\n", MaxClass[i], fMaxProb[i]);
12     }
13 }
```

对输出结果进行后处理，提取 Top-5 分类结果。

步骤：

1. 把输出数据转为 float*
2. 调用 rknn_GetTop() 提取前 5 个最高概率
3. 打印类别索引和概率

2.3.12 释放资源

代码块

```
1  rknn_outputs_release(ctx, 1, outputs);
2  if (ctx > 0) {
3      rknn_destroy(ctx);
4  }
5  if (model) {
6      free(model);
7  }
8  return 0;
```

理所有分配的资源，防止内存泄漏。

- rknn_outputs_release(): 释放输出缓冲区

- `rknn_destroy()`: 销毁 RKNN 上下文
- `free(model)`: 释放模型内存